

2025 Fall CSED451 Assignment 3-2 Report

Team Name: openGameLab

Team Member #1: 안재영

컴퓨터공학과/20220019/enter21

Team Member #2: 정윤희

컴퓨터공학과/20240385/jyhyeok

1. Technical Details

개발 환경: Visual Studio 2022 버전 17.14.9 (July 2025)

cpp 확장자 파일을 powershell 스크립트 파일로 만든 후 powershell에서 빌드하여 구현 결과를 확인할 수 있다. 또한, OpenGL을 사용하는 코딩에 익숙해야 한다.

2. Implementation Details

2-1. Outline

Assignment3-1은 Assignment1, Assignment2, Assignment3-1에서 구현했던 Bullet Hell shooter의 모든 기능들 및 그래픽을 유지하면서, OpenGL legacy pipeline을 모두 OpenGL Shader Language로 교체하는 것이 주 목표이다. 플레이어, 적, bullet 등 텍스트를 제외한 모든 엔티티가 GLSL로 표현되었으며, 제공된 assets 파일의 3D 모델들을 적극적으로 활용했다. 이를 위해 Shader 클래스와 shader.vs, shader.fs 파일을 추가했으며, 기존의 Model 클래스를 VAO와 VBO를 사용하는 방식으로 변경했다.

glm 라이브러리를 include하여 기존 OpenGL legacy pipeline 방식 대신, 직접 모든 행렬을 계산할 수 있도록 환경을 세팅했다. 변환 행렬은 4x4행렬로 저장되어, 전역변수로 프로그램 전체에서 변경될 수 있으며 적절히 사용된다. 이에 따라 기존 draw함수들과, 그래픽 스타일 및 카메라 뷰 설정 함수들을 적절히 변경했다.

Q키를 눌러 그래픽 스타일을 바꿀 수 있다. 이번 과제에서는 기존에 구현되어 있던 opaque polygon style과 wireframe style 두 가지 그래픽 스타일에 더해, wireframe style with hidden line removal을 구현한다. 이를 위해 setGraphicStyleThenRender라는 함수를 추가하여 currentGraphicStyle이라는 변수에 따라 그래픽 스타일을 바꾸도록 했으며, 이 함수에서 렌더링까지 진행한다. 이를 위한 관련 키 상호작용을handleKeyDown 함수에

구현했다.

3D 그래픽 방식과 다양한 카메라 뷰 추가로 인해, 기존에 사용하던 윈도우 창을 제한 구역으로 하기에는 플레이어가 이동 제한 구역을 잘 인지하지 못한다는 문제가 발생한다. 따라서 과제 문서에 따라 플레이어가 움직일 수 있는 공간인 bounding box를 구현하였으며, 노란색 선으로 구역을 잘 보이게 하였다. 이 bounding box 또한 씬 그래프에 의해 노드로 관리되어 그려진다. Bullet들도 이 bounding box 외부로 이동하면 사라진다. 이 bounding box에도 VAO를 적용하였고, 그 기능은 InitBoundingBox 함수가 맡는다.

2-2. Additional Requirements

- 적 색과 HP 연동

Enemy 객체의 health와 maxHealth 값을 가져와 현재 남은 체력의 비율을 계산하고, 선형 보간 함수 lerp를 사용해 healthyColor와 damagedColor 사이에서 비율에 맞는 중간 색을 계산한다. 함수에 의해 반환된 색상 값을 적의 color 속성에 덮어씌워서 HP의 변화를 색깔로 인지할 수 있도록 한다.

- 적의 플레이어 타겟팅

Enemy::update 함수에서 매 프레임마다 적의 현재 위치를 기반으로 플레이어와의 x, y 좌표의 차이를 계산한다. 그 후 atan2 함수를 통해 적이 플레이어를 향하는 벡터의 각도를 계산하고, Enemy 구조체 내의 targetAngle에 저장한다. 적을 실제로 그리는 updateSceneGraph 함수에서 enemyNode->rot.y에 해당 각도를 대입하여 drawRecursive가 해당 노드를 그릴 때 플레이어를 향한 방향으로 정렬되게 구현하였다.

- 플레이어 총알 명중 파티클

handleCollision 함수가 플레이어 총알과 적의 충돌을 감지한 순간에 총알이 맞은 위치에서 8개 방향으로 작은 노란색 파티클이 퍼져나가도록 한다. 이를 위해 위치, 속도, 생존 시간과 색깔 인자를 가지는 구조체 Particle을 새롭게 추가한다.

생성 함수 bulletParticleEffect는 파티클을 생성할 피격 위치 x, y를 인자로 받으며, 이를 기준으로 8방향으로 퍼져나가는 원형 파티클들을 생성해 velocity와 lifetime을 할당한다. 이 때 lifetime에는 rand()를 통한 약간의 랜덤성을 부여해 연출의 생동감을 부여한다. 생성된 파티클은 particleNodePool에 저장되며, updateParciels에 의해 관리된다.

updateParticles는 timer에 의해 호출되어 매 프레임마다 particles 속 파티클들의 위치를 속도를 기반으로 갱신하고, 생존 시간을 감소시키다가 수명이 다할 경우 삭제한다. 3-1의 구현에서는 함수 drawParticleEffect를 사용해 파티클 효과를 그렸지만, 이번에는 다른 오브젝트들과 마찬가지로 Node 속 순회에 의해 접근하여 그린다.

- 플레이어 추진 파티클

플레이어가 W키를 통해 앞으로 추진할 때 기체 뒤쪽에서 파티클이 분출되도록 한다. 이 파티클들은 boostParticleEffect 함수를 통해 생성되며, processInput의 'w' 입력 인식 부분에서 호출된다. 플레이어 제트기 모델의 후방 위치에서 1~2개의 파티클을 생성해 particles에 추가하고, 파티클의 속도는 플레이어의 뒤쪽을 향하도록 마찬가지로 rand()를 통해 약간의 랜덤성을 준다. 파티클의 관리는 위의 총알 명중 효과와 동일한 함수들에 의해 처리된다.

- 적 주위 공전 엔티티

게임의 3D 입체감을 조금 더 살릴 수 있도록, 적의 주위를 공전하는 엔티티를 구현하여 게임이 단순 평면만을 그리고 있는 것이 아님을 강조하였다. 이를 위해 씬 그래프의 관계에서 적의 자식에 새로운 노드를 추가하였고, main 함수에서 이를 위한 enemyOrbitNodePool을 생성하였다.

updateSceneGraph 함수에서 enemyNode를 enemiesGrounNode에 추가하기 전에 enemyOrbitNodePool에서 공전 엔티티 노드를 가져와 children.push_back()으로 enemyNode의 자식으로 추가하고, glGet(GLUT_ELAPSED_TIME)을 인자로 삼각함수를 호출해 공전 엔티티들의 x, y좌표에 대입하였다. 이를 통해 평면을 뚫고 각 enemyNode의 로컬 XY 평면을 공전하는 4개의 구체를 구현하였다.

2-3. Design Rationale

이번 과제에서는 OpenGL legacy pipeline 대신 GLSL을 사용하여 셰이더와 VAO, VBO 개념을 도입하였다. GPU를 사용하게 됨으로써 이후 최적화 및 더 많고 복잡한 모델들 그리기를 기대할 수 있게 되었다. 이번 과제에서 추가되거나 변경된 사항은 다음과 같다.

1) GLSL 도입: vertex shader, fragment shader, Shader 클래스를 구현했고 .vs, .fs 파일을

로드하여 사용한다. VAO, VBO를 사용하여 행렬을 포함한 데이터들을 GPU로 보낸다. 또한 GLSL을 사용함으로써, 기존 glBegin 등의 OpenGL legacy pipeline을 사용하는 코드들을 삭제했다.

2-4. Design Implementation

Shader 클래스에서는 .vs, .fs 확장자 파일을 읽어온 후, 셰이더를 컴파일하고, 셰이더와 프로그램 링크한다. 또한 use, setMat4, setVec3 함수들을 선언하여 셰이더 활성화와 uniform 변수 위치 검색 및 값 할당을 편하게 할 수 있게 구현했다. 전역변수로 셰이더 포인터를 추가하고, main함수 내부에서 셰이더를 로드하여 활성화시킨다.

기존의 Model 클래스에 glGenVertexArrays, glGenBuffers, glBindVertexArray, glBindBuffers, glBufferData, glVertexAttribPointer 등을 이용하여 VAO와 VBO를 사용하여 데이터를 GPU에 보내는 기능을 추가로 구현했다.

glm 관련 라이브러리를 include하여, 행렬 변환 및 포인터 변환 기능을 사용할 수 있게 하였으며, viewMatrix와 projMatrix를 glm::mat4 타입의 전역 변수로 선언하였다. main에서 glm::value_ptr을 이용하여 포인터를 넘긴다.

기존 setGraphicStyle, setCameraViews 함수를 행렬을 이용하여 glm 버전으로 적절히 변경한다. 특히 setGraphicStyle 함수는 각 그래픽 스타일마다 적절히 그리기 위해, 렌더 기능까지 합쳐 setGraphicStyleThenRender라는 새로운 함수로 구현했다.

그래픽 스타일 중 이번 과제에 새로 추가해야 하는 wireframe style with hidden line removal은, 깊이 버퍼를 이용하여 검정색 면을 그린 후 그 다음에 선을 그려서, 검정색 면보다 뒤에 있는 선은 보이지 않게 하는 방식으로 구현할 수 있다. glPolygonOffset, glColorMask 등을 이용하면 된다.

Bounding box를 그리기 위한 데이터도 VAO, VBO로 미리 GPU에 보내 둔다. 선 12개, 즉 점 24개의 데이터가 필요하며 이는 InitBoundingBox 함수에서 관리한다.

2-5. Additional Background

본 구현을 위해서는 OpenGL에서 3D 공간을 표현하는 방법, 그리고 씬 그래프 자료 구조 및 GLSL에 대한 전반적인 이해가 필요했다.

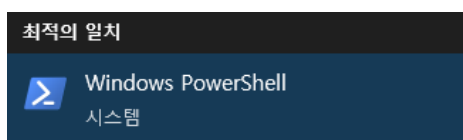
1) 3D 카메라 변환: 3D 공간을 2D 공간으로 표현하기 위해 두 가지 핵심적인 행렬 변환을 활용하였다. 첫번째는 카메라의 위치와 시점 방향을 설정하는 시점 변환 `gluLookAt`, 두번째는 3D 공간에 원근감을 부여하는 투영 변환 `gluPerspective`와 `glOrtho`였다. 이들을 구분해 적절하게 사용하면서 3D 공간을 바라보는 3가지 시점을 구현할 수 있었다.

2) 계층적 씬 그래프: 엔티티들을 Node 그래프로 관리하기 위해 씬 그래프 자료구조를 활용하였다. 이 트리를 `drawRecursive` 함수가 `glPushMatrix`와 `glPopMatrix`를 사용해 행렬 상태를 관리하면서 재귀적으로 순회하며, 따라서 플레이어/적의 로컬 좌표계에서 공전하는 엔티티 등 계층적 움직임을 쉽게 구현할 수 있었다.

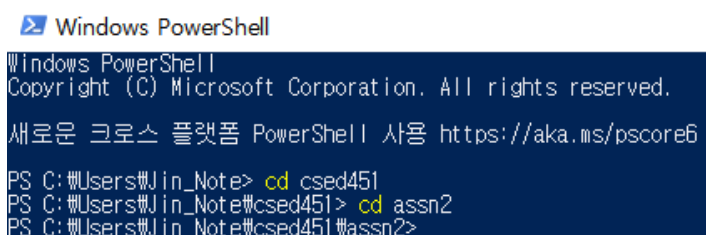
3) GLSL: 이번 구현에서는 OpenGL의 legacy pipeline을 모두 Shading Language (GLSL)로 대체하며 렌더링 파이프라인을 변경하였다. Vertex Shader와 Fragment Shader를 컴파일 및 링크 함으로써 Shader 프로그램을 활성화하였고, `Node::drawRecursive` 함수의 씬 그래프 순회 과정에서 모델 행렬, 뷰 행렬, 투영 행렬 및 오브젝트들의 색상을 셰이더의 Uniform 변수로 전송한다. 이 변수들은 GPU로 전달되고, GPU는 이 행렬들을 효율적으로 곱해 각 정점의 최종 화면 좌표 `gl_Position`을 계산하고 Fragment Shader가 `objectColor`를 받아 픽셀을 칠함으로써 렌더링 과정이 완료된다.

3. End-User Guide

Windows powershell을 실행한다.



프로젝트가 저장된 파일 위치로 이동한다.



.\build.ps1을 입력한다.

```
PS C:\Users\Jin_Note\csed451\assn2> .\build.ps1

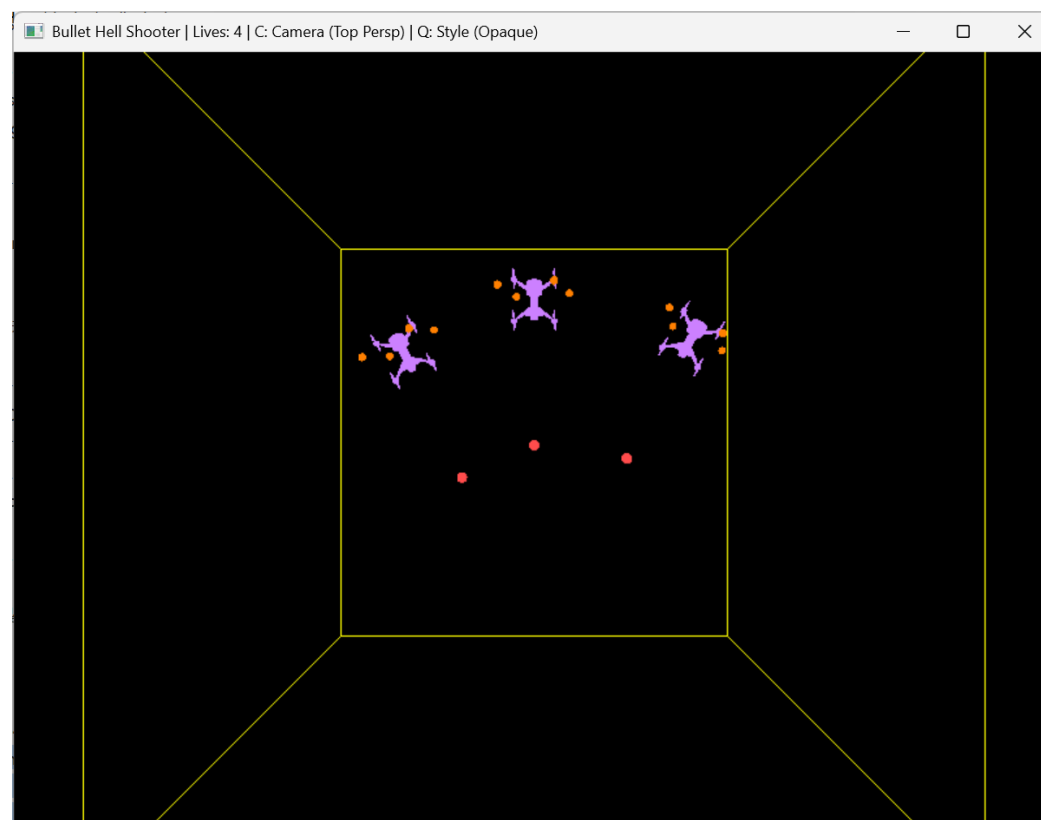
디렉터리: C:\Users\Jin_Note\csed451\assn2

Mode                LastWriteTime         Length Name
----                -
d-----          2025-11-07 오후 2:14             build
*****
** Visual Studio 2022 Developer PowerShell v17.14.9
** Copyright (c) 2025 Microsoft Corporation
*****
Microsoft (R) C/C++ 최적화 컴파일러 버전 19.44.35213(x64)
Copyright (c) Microsoft Corporation. All rights reserved.

code.cpp
Microsoft (R) Incremental Linker Version 14.44.35213.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:C:\Users\Jin_Note\csed451\assn2\build\build.exe
/LIBPATH:C:\Users\Jin_Note\csed451\assn2\lib
freeglut.lib
glew32.lib
opengl32.lib
C:\Users\Jin_Note\csed451\assn2\build\code.obj
Execution file build successfully. Running...
```

잘 실행되는 것을 확인할 수 있다.



실행된 게임에서는 WASD 키를 통해 플레이어 이동이 가능하며, 스페이스바를 눌러 총알을 발사할 수 있다. Q키를 누르면 그래픽 스타일을, C키를 누르면 카메라 뷰 스타일을 변경할 수 있다. 현재 카메라 및 그래픽 스타일은 윈도우 창 이름에 표시된다.

프로그램 종료 시 powershell 창에는 아래 메시지가 출력된다.

```
Execution exited.  
PS C:\Users\Jin_Note\csed451\assn2>
```

이후 다시 빌드하여 프로그램을 실행시키는 것 또한 가능하다.

만약 Powershell 스크립트 실행 정책이 제한되어 있는 등의 이유로 Powershell에서 스크립트를 실행할 수 없다면, Get-ExecutionPolicy 명령어를 입력하여 실행 정책이 제한되어 있는지 확인해야 한다. 만약 그렇다면, Set-ExecutionPolicy RemoteSigned -Scope CurrentUser 명령어를 입력하여 스크립트 실행 정책을 변경해야 한다. 정책 여부 변경을 묻는 메시지가 나타나면 Y를 입력하여 스크립트 실행 정책을 변경하면 실행이 잘 된다.

```
PS C:\Users\Jin_Note> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser  
실행 규칙 변경  
실행 정책은 신뢰하지 않는 스크립트로부터 사용자를 보호합니다. 실행 정책을 변경하면 about_Execution_Policies 도움말  
원문(https://go.microsoft.com/fwlink/?LinkID=135170)에 설명된 보안 위험에 노출될 수 있습니다. 실행 정책을  
변경하시겠습니까?  
[Y] 예(Y) [A] 모두 예(A) [N] 아니요(N) [L] 모두 아니요(L) [S] 일시 중단(S) [?] 도움말 (기본값은 "N"): Y  
PS C:\Users\Jin_Note>
```

4. Discussions/Conclusions

4-1. Obstacles

그래픽 스타일 중 wireframe style with hidden line removal의 경우, bounding box가 선으로 그려져서 보이지 않게 되는 문제가 있었다. 이는 기본값이 GL_LESS여서 그랬던 것으로, GL_EQUAL로 바꾸니까 해결되었다.

4-2. Improvement Idea

Assignment 문서에서 제시되었던 Destructible enemy parts를 추가적으로 구현하면 게임 플레이어가 조금 더 슈팅 게임이라는 게임 장르에 몰입할 수 있을 것이다. 또한 현재의 움직임은 굉장히 단순한데, 공격 패턴이나 이동 패턴 등을 더 다양하게 설계한다면 게임성을 높일 수 있다.

4-3. Lessons

GLSL과 프로그램 전체에서 쓰이는 행렬을 이용하여 3D 그래픽을 구현하는 방법을 과제를 통해 직접 구현하며 체화할 수 있었다.

5. References

Learn OpenGL - <https://learnopengl.com/>

6. AI-assisted Coding References

이번 과제의 약 60%가 AI(Gemini)의 도움을 받아 구현되었다. 이전 과제 3-1에서 구현한 부분들에서 legacy pipeline을 활용해 수정해야 하는 부분들을 찾은 후, GLSL 방식으로 수정하는 데에 AI의 도움을 받았다.

7. individual contribution

이번 프로젝트는 두 팀원이 50:50 비율로 각자의 역할을 나누어 충실히 수행하였다. 이전 과제인 3-1을 기반으로 이번 구현이 진행되었던 만큼, 각자가 3-1에서 담당했던 부분을 동일하게 GLSL 방식으로 구현하도록 역할을 분담하였다.